CareerTwin AI

Sistema de análisis de habilidades profesionales y detección orientativa de ofertas fraudulentas

Informe final de proyecto de Inteligencia Artificial

Estudiantes

Dayana Sofía Mendes Perafán

Andrea Catalina Cortes

Sergio Andres Perez Ramirez

Docente

José Miguel Llanos Mosquera

Corporación Universitaria del Huila – CORHUILA

Facultad de Ingeniería

Programa de Ingeniería de Sistemas

Asignatura: Inteligencia Artificial

Semestre 2026-2

Neiva, Huila · 2026

Resumen

CareerTwin AI es un prototipo académico de procesamiento de lenguaje natural (NLP) que identifica habilidades profesionales mediante una red neuronal convolucional (CNN) multietiqueta y ofrece orientación sobre coincidencias y brechas de habilidades. Integra, además, Fraud Shield, un clasificador complementario que estima señales de riesgo en textos de ofertas laborales. El estudio aplicó auditoría y limpieza de datos, control de descripciones repetidas, partición de conjuntos independientes, entrenamiento supervisado e integración de los modelos con Flask. La CNN se entrenó durante 30 épocas sobre una muestra reproducible de 50.000 ofertas únicas del conjunto LinkedIn Jobs & Skills 2024, con 100 etiquetas. Su evaluación final en 7.500 ofertas de prueba reportó Micro-F1 de 0,3443 y Macro-F1 de 0,1343. Para Fraud Shield se utilizaron 15.662 registros depurados del conjunto EMSCAD; el clasificador TF-IDF y regresión logística obtuvo precisión de 0,8090, recall de 0,6990 y F1 de 0,7500 en la clase fraudulenta. La interfaz web muestra habilidades detectadas, compatibilidad orientativa, recomendaciones, tres gráficas y el análisis de riesgo. La validación final registró 34 pruebas automatizadas aprobadas. La solución se compartió mediante acceso HTTPS temporal; su uso productivo requeriría infraestructura permanente y validación adicional.

Palabras clave: habilidades, empleo, convolución, fraude, clasificación.

Abstract

CareerTwin AI is an academic natural language processing (NLP) prototype that detects professional skills through a multilabel convolutional neural network (CNN) and provides guidance on skill matches and gaps. It also integrates Fraud Shield, a supplementary classifier that estimates potential risk signals in job postings. The study followed a workflow involving data auditing, text cleaning, duplicate control, independent data splitting, supervised learning, and Flask integration. The CNN was trained for 30 epochs using a reproducible sample of 50,000 unique postings from the LinkedIn Jobs & Skills 2024 dataset with 100 skill labels. On 7,500 test postings, its Micro-F1 was 0.3443 and its Macro-F1 was 0.1343. Fraud Shield used 15,662 cleaned EMSCAD records; its TF-IDF and logistic regression pipeline achieved precision of 0.8090, recall of 0.6990, and F1 of 0.7500 for the fraudulent class. The interface supports profile analysis, skill gaps, recommendations, three charts, and fraud-risk evaluation. A final validation run passed 34 automated tests. Remote access was enabled through a temporary HTTPS tunnel; production use would require permanent hosting and further independent validation.

Keywords: skills, employment, convolution, fraud, classification.

Tabla de Contenido

Resumen 2
Abstract 3
1. Introducción 7
2. Planteamiento del problema y justificación 8
3. Objetivos 9
4. Marco conceptual y metodología 10
5. Fuentes de datos, auditoría y preparación 11
6. Arquitectura y entrenamiento de la CNN 14
7. Fraud Shield: preparación, modelo y evaluación 16
8. Integración de la aplicación web 19
9. Resultados y validación 25
10. Despliegue, reproducibilidad y limitaciones 27
11. Conclusiones 30
12. Recomendaciones 31
Referencias 31
Anexo A. Evidencias visuales del desarrollo 32
Anexo B. Comprobaciones para la sustentación 34

Índice de tablas

Tabla 1. Distribución de los datos de LinkedIn 12
Tabla 2. Características de la CNN 14
Tabla 3. Comparación de indicadores CNN 15
Tabla 4. Preparación y partición de Fraud Shield 17
Tabla 5. Métricas del modelo de fraude 18

Orientación para la lectura de las tablas

Las cinco tablas permiten seguir las decisiones centrales del proyecto. La primera identifica la distribución de las 50.000 ofertas de LinkedIn usadas en el clasificador multietiqueta. La segunda resume las capas, parámetros y configuración de la red convolucional. La tercera presenta métricas sobre un conjunto de prueba independiente; por ello sus resultados no deben interpretarse como rendimiento sobre los ejemplos vistos durante el entrenamiento.

Las Tablas 4 y 5 corresponden al componente Fraud Shield. La Tabla 4 documenta la depuración del corpus EMSCAD y sus particiones; la Tabla 5 contrasta precisión, exhaustividad y F1 en validación y prueba. Dado que las ofertas fraudulentas son minoritarias, la interpretación debe centrarse en esas métricas y en los errores de clasificación, no solamente en la proporción total de aciertos.

Índice de figuras

Figura 1. Secuencia de trabajo de CareerTwin AI. 11
Figura 2. Partición del conjunto multietiqueta (50.000 ofertas). 13
Figura 3. Arquitectura neuronal implementada para clasificación multietiqueta. 14
Figura 4. Métricas de la CNN en el conjunto de prueba. 16
Figura 5. Matriz de confusión de Fraud Shield sobre 2.350 ofertas. 18
Figura 6. Análisis personalizado de habilidades en la interfaz de CareerTwin AI. 20
Figura 7. Brechas profesionales y coincidencias con familias laborales. 21
Figura 8. Visualización 1: compatibilidad entre un perfil de ejemplo y familias laborales. 22
Figura 9. Visualización 2: similitud entre familias y transiciones profesionales. 23
Figura 10. Visualización 3: demanda de ofertas por familia y habilidades frecuentes. 24
Figura 11. Interfaz integrada de Fraud Shield con una oferta ficticia de riesgo. 25
Figura 12. Resultado de la ejecución final de pruebas automatizadas. 26
Figura 13. Evidencia de revisión y merge de un Pull Request en GitHub. 27
Figura 14. CareerTwin AI accesible mediante enlace HTTPS temporal de Cloudflare. 28
Figura 15. Evidencia de trabajo colaborativo: publicación de una rama de Flask en GitHub. 33
Figura 16. Evidencia del entorno de desarrollo y revisión de código. 33
Figura 17. Recomendaciones profesionales generadas para el perfil de prueba. 34

Orientación para la lectura de las figuras

Las figuras recogen tanto resultados calculados como evidencias de uso del prototipo. Las Figuras 1 a 4 describen el proceso de construcción, la división de datos, la arquitectura y las métricas finales de la CNN. La Figura 5 representa los errores y aciertos de Fraud Shield; las Figuras 6 y 7 documentan la detección de competencias y las brechas observadas en una prueba funcional.

1. Introducción

La búsqueda de empleo y la planificación de una trayectoria profesional requieren comprender qué competencias se poseen, qué capacidades solicitan los sectores productivos y cuáles son las brechas que conviene cerrar. Las ofertas laborales contienen información valiosa, aunque se presentan principalmente en texto libre y varían en estructura, idioma y calidad. Por otra parte, la existencia de ofertas potencialmente fraudulentas introduce riesgos que exigen una evaluación cuidadosa y no solo una lectura superficial.

En este contexto, el equipo desarrolló CareerTwin AI como proyecto final de la asignatura Inteligencia Artificial de la Corporación Universitaria del Huila (CORHUILA; Corporación Universitaria del Huila, 2026). Se optó por una arquitectura NLP que procesa descripciones de empleo con una red neuronal convolucional (CNN) y complementa sus predicciones con reglas de reconocimiento textual, matrices de habilidades por familia ocupacional y un módulo supervisado para señales de fraude.

El documento presenta las etapas realizadas, desde la auditoría de las fuentes de datos hasta la integración de la interfaz, la evaluación cuantitativa y las verificaciones finales. Se distinguen los resultados de modelos realmente entrenados de los perfiles de demostración del panel general. También se deja constancia de las funciones que requieren desarrollo adicional antes de considerar el sistema apto para un escenario productivo.

2. Planteamiento del problema y justificación

2.1. Problema

Las personas que describen su experiencia profesional no siempre disponen de herramientas que transformen esa descripción en una lista interpretable de competencias y áreas de mejora. Asimismo, comparar manualmente un perfil con varias familias laborales es laborioso, pues exige revisar muchos requisitos y resolver sinónimos o variantes de escritura. En paralelo, la identificación de ofertas engañosas es compleja por la escasez relativa de ejemplos fraudulentos y por la variación de los mensajes utilizados.

2.2. Pregunta orientadora

¿Cómo aprovechar el procesamiento de lenguaje natural y el aprendizaje automático para identificar habilidades en textos profesionales, ofrecer orientación laboral explicable y advertir sobre posibles señales de fraude en publicaciones de empleo?

2.3. Justificación

La relevancia académica se relaciona con la implementación completa de un flujo de inteligencia artificial: obtención de datos abiertos, limpieza y particionado, entrenamiento de una red neuronal, evaluación con métricas adecuadas, creación de una API y visualización de resultados en una aplicación web. Su valor práctico radica en mostrar coincidencias y brechas comprensibles, sin reemplazar la valoración humana ni emitir conclusiones deterministas acerca de la empleabilidad o legitimidad de una oferta.

3. Objetivos

3.1. Objetivo general

Diseñar, entrenar e integrar una solución web basada en NLP y redes neuronales que identifique habilidades profesionales a partir de texto y presente análisis orientativos de compatibilidad laboral, complementados con detección de posibles señales de fraude en ofertas de empleo.

3.2. Objetivos específicos

Auditar y preparar fuentes de datos abiertas, preservando los términos técnicos y evitando duplicados entre entrenamiento, validación y prueba.

Construir una CNN multietiqueta para la identificación de 100 habilidades y evaluar su rendimiento sobre un conjunto independiente de prueba.

Implementar un clasificador complementario TF-IDF + regresión logística para señales de fraude, controlando el desbalance de clases.

Integrar modelos, cálculo de brechas y visualizaciones en una aplicación web Flask accesible mediante endpoints HTTP.

Validar el comportamiento funcional con pruebas automatizadas y registrar limitaciones, evidencias y posibles mejoras.

4. Marco conceptual y metodología

4.1. Conceptos fundamentales

El procesamiento de lenguaje natural (NLP) reúne técnicas que permiten transformar texto en representaciones analizables por algoritmos. En este proyecto se utilizó clasificación multietiqueta, siguiendo los fundamentos del aprendizaje profundo aplicados a textos (Chollet, 2021; Kim, 2014): una misma oferta puede relacionarse con varias habilidades, por ejemplo Python, SQL, comunicación y trabajo en equipo. La CNN aplica convoluciones sobre representaciones de palabras para identificar patrones locales de texto; su capa final emplea funciones sigmoid independientes en lugar de una softmax exclusiva.

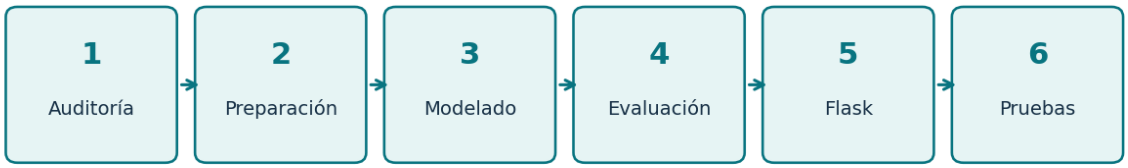
TF-IDF pondera vocablos según su frecuencia en el documento y su rareza en el conjunto de textos, una representación empleada con clasificadores implementados en Scikit-learn (Pedregosa et al., 2011). Acoplado con regresión logística, permite estimar una puntuación útil para clasificación binaria. En ambas tareas, precisión, recall y F1 expresan propiedades distintas de la evaluación: los errores no pueden reducirse a una única cifra de exactitud, especialmente cuando las clases presentan desbalance.

4.2. Metodología aplicada

El trabajo siguió un ciclo iterativo alineado con fases de comprensión, preparación, modelado, evaluación e implementación que pueden relacionarse con CRISP-DM. Se documenta este esquema como marco descriptivo de los pasos realmente ejecutados; la evidencia principal son los scripts de preparación, reportes versionados, solicitudes de integración y pruebas. El

código se desarrolló en Visual Studio Code y se gestionó colaborativamente mediante ramas y Pull Requests de GitHub.

Figura 1
Secuencia de trabajo de CareerTwin AI.



Nota. Elaboración propia a partir de las fases implementadas en el repositorio.

4.3. Organización del equipo

Sofía coordinó la arquitectura y las ejecuciones de entrenamiento/evaluación, la integración de servicios de IA y el cierre de pruebas; Catalina lideró auditoría, calidad de datos, normalización y validación de habilidades y fraude; Sergio desarrolló e integró la interfaz y los servicios Flask, incluidos los resultados, gráficas y la experiencia de usuario. El trabajo se incorporó a la rama main mediante revisiones y Pull Requests. Las funciones fueron complementarias y todos participaron en las verificaciones finales.

5. Fuentes de datos, auditoría y preparación

5.1. Dataset de ofertas de LinkedIn

Se utilizó el conjunto abierto 1.3M LinkedIn Jobs & Skills 2024, publicado en Kaggle (Asaniczka, 2024). La auditoría contó 1.348.454 registros en linkedin_job_postings.csv,

1.296.381 en job_skills.csv y 1.297.332 en job_summary.csv. Los archivos se unieron mediante job_link. Se identificaron 1.294.296 ofertas con las tres tablas y datos de habilidades; tras filtrar resúmenes de menos de 50 palabras quedaron 1.290.647 candidatas, de las cuales se contabilizaron 934.563 textos únicos.

Se eliminaron o agruparon duplicados exactos y variantes de formato; para ello se protegieron términos técnicos que dependen de signos, como C++, C#, .NET y Node.js. Se estandarizaron habilidades, correos y enlaces y se seleccionó una muestra reproducible de 50.000 descripciones únicas con semilla 42. El vocabulario piloto abarcó 100 habilidades y las etiquetas de copias equivalentes se consolidaron por unión, con un registro de soporte de aparición.

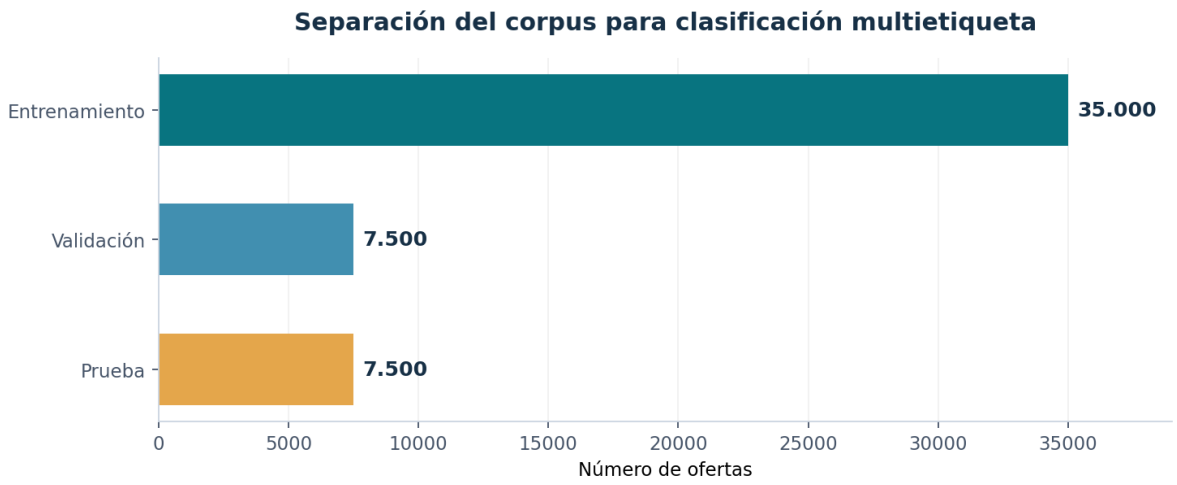
Tabla 1

Distribución de los datos de LinkedIn

Partición	Registros	Porcentaje	Uso
Entrenamiento	35.000	70 %	Ajustar parámetros
Validación	7.500	15 %	Elegir umbral y mejor modelo
Prueba	7.500	15 %	Evaluar resultados finales

Figura 2

Partición del conjunto multietiqueta (50.000 ofertas).



Nota. Elaboración propia a partir de reports/particiones_resumen.json. División por grupos de descripción; cero grupos cruzados.

Las particiones se generaron por grupos equivalentes y se comprobó que ningún grupo apareciera en más de una división. La auditoría encontró 7.760 registros sin etiquetas del vocabulario de 100 habilidades, lo que limita la cobertura de un entrenamiento supervisado. De las menciones totales de habilidades normalizadas, ese vocabulario cubrió aproximadamente el 20,57 %, mientras que la lista revisada más extensa elevaría la cobertura a alrededor del 25,49 %. Estas cifras explican parte de las dificultades de generalización de la CNN.

5.2. Calidad de datos y visualización de mercado

El equipo produjo tablas agregadas de 23 familias laborales, relaciones de habilidades y transiciones entre familias. Se verificaron recuentos, sumas y ejemplos de compatibilidad mediante controles independientes. Las gráficas generales del dashboard proceden de datos procesados y perfiles de demostración; los ejemplos no deben interpretarse como personas reales ni como una predicción personalizada de contratación.

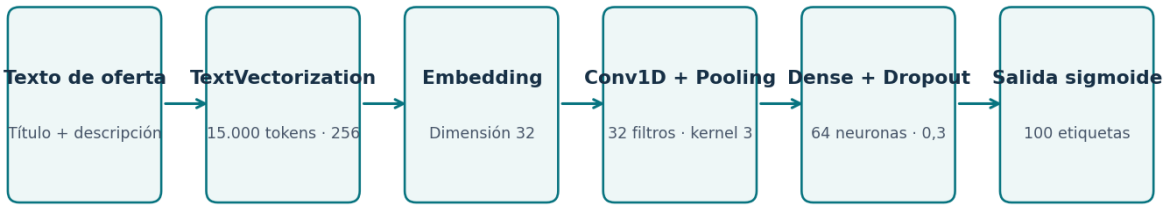
6. Arquitectura y entrenamiento de la CNN

6.1. Diseño de la red neuronal

La entrada del modelo concatena el título y la descripción limpia de cada oferta. Se utilizó TextVectorization con un vocabulario máximo de 15.000 tokens y longitud de secuencia de 256, seguido de Embedding de dimensión 32, una capa Conv1D de 32 filtros y kernel 3 con ReLU, GlobalMaxPooling1D, Dense de 64 neuronas y Dropout de 0,3. La capa de salida contiene 100 neuronas sigmoid, una por habilidad. La función de pérdida fue binary crossentropy y el optimizador Adam con tasa de aprendizaje de 0,001.

Figura 3

Arquitectura neuronal implementada para clasificación multietiqueta.



Nota. Elaboración propia a partir de src/train_skills.py; esquema simplificado de las capas de Keras.

Tabla 2

Características de la CNN

Elemento	Configuración
Tarea	Clasificación multietiqueta de 100 habilidades
Datos de entrenamiento / validación	35.000 / 7.500 ofertas
Tokenización	15.000 tokens máximos; longitud 256
Representación y convolución	Embedding 32; Conv1D 32 filtros, kernel 3
Regularización	Dropout 0,30
Optimización	Adam (0,001); binary crossentropy
Entrenamiento	30 épocas; semilla 42

Elemento	Configuración
Selección	Mejor val_loss; época 6
Umbral de inferencia	0,20 elegido en validación

6.2. Proceso de entrenamiento y selección

El entrenamiento se ejecutó durante 30 épocas con lotes de 32, barajado de entrenamiento y registro del historial. Se guardó la mejor versión de la red mediante ModelCheckpoint sobre la pérdida de validación, en lugar de asumir que la última época era la mejor. La mejor época fue la sexta y alcanzó una pérdida de validación de 0,1395. El umbral de predicción se seleccionó con la partición de validación y la evaluación final se realizó sobre las 7.500 ofertas de prueba.

El modelo se exportó a models/career_skills.keras y se complementó en producción con un detector textual de habilidades. Dado que el entrenamiento se apoyó en etiquetas extraídas automáticamente de ofertas, los resultados reflejan tanto el aprendizaje de la red como el ruido y los sesgos del dataset original. No se deben interpretar las puntuaciones sigmoid como probabilidades calibradas de que una persona posea una competencia.

6.3. Evaluación del modelo neuronal

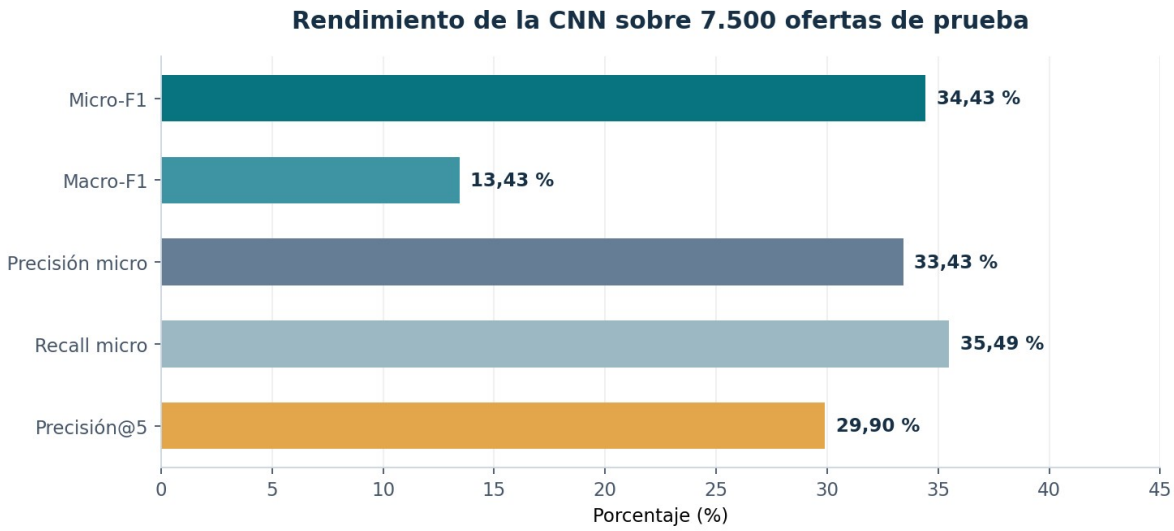
Tabla 3

Comparación de indicadores CNN

Indicador	Valor en prueba	Interpretación
Micro-F1	0,3443	Balance global entre precisión y recall
Macro-F1	0,1343	Promedio por habilidad; evidencia debilidad en clases raras
Precisión micro	0,3343	Proporción de aciertos entre predicciones positivas
Recall micro	0,3549	Cobertura de etiquetas positivas
Precision@5	0,2990	Calidad de las cinco habilidades priorizadas

Figura 4

Métricas de la CNN en el conjunto de prueba.



Nota. Elaboración propia con valores de reports/evaluacion_final_skills.json. Las barras no representan precisión laboral ni probabilidades de contratación.

El Micro-F1 de 0,3443 y, en especial, el Macro-F1 de 0,1343 indican rendimiento moderado o limitado para un problema multietiqueta complejo. La caída del promedio macro sugiere dificultades con categorías menos representadas. Entre las causas documentadas se encuentran la concentración de etiquetas frecuentes, la ausencia de habilidades en algunos registros y la necesidad de adaptar los textos a perfiles personales, que no son idénticos a las ofertas con las que se entrenó la red.

7. Fraud Shield: preparación, modelo y evaluación

7.1. Auditoría y preparación de EMSCAD

El módulo complementario utilizó EMSCAD (Real or Fake Job Posting Prediction), con 17.880 ofertas originales, de las cuales 866 estaban etiquetadas como fraudulentas (4,84 %). La auditoría identificó desbalance pronunciado, registros duplicados, campos faltantes y marcadores

de correo, URL y teléfono provenientes del corpus de origen. Para evitar diferencias entre entrenamiento y uso real, el pipeline aplica la misma normalización al texto de usuario y a los registros de entrenamiento, sustituyendo marcadores por [EMAIL], [URL] y [PHONE].

Se emplearon únicamente campos textuales compatibles con el formulario de la aplicación: título, descripción, requisitos y beneficios. Se excluyeron identificadores y metadatos que pudieran proporcionar señales de fraude artificiales o no disponibles en tiempo de inferencia. Tras los filtros y eliminación de 2.217 copias repetidas, quedaron 15.662 ofertas únicas, sin grupos con etiquetas contradictorias, divididas estratificadamente en entrenamiento, validación y prueba.

Tabla 4

Preparación y partición de Fraud Shield

Conjunto	Total	Fraudulentas	No fraudulentas
Original	17.880	866	17.014
Después de depuración	15.662	684	14.978
Entrenamiento	10.963	479	10.484
Validación	2.349	102	2.247
Prueba	2.350	103	2.247

7.2. Entrenamiento del clasificador secundario

Fraud Shield no utiliza la CNN principal: implementa un pipeline de Scikit-learn compuesto por TfidfVectorizer con hasta 50.000 características y unigramas/bigramas, seguido de regresión logística con class_weight="balanced" y solver liblinear. El vectorizador se ajustó únicamente al conjunto de entrenamiento. El umbral se eligió sobre validación en una búsqueda de 0,10 a 0,90, con pasos de 0,05, maximizando F1 y usando recall como criterio de desempate; el valor final fue 0,60. Así se evitó adaptar las decisiones del modelo a la partición de prueba.

7.3. Resultados y matriz de confusión

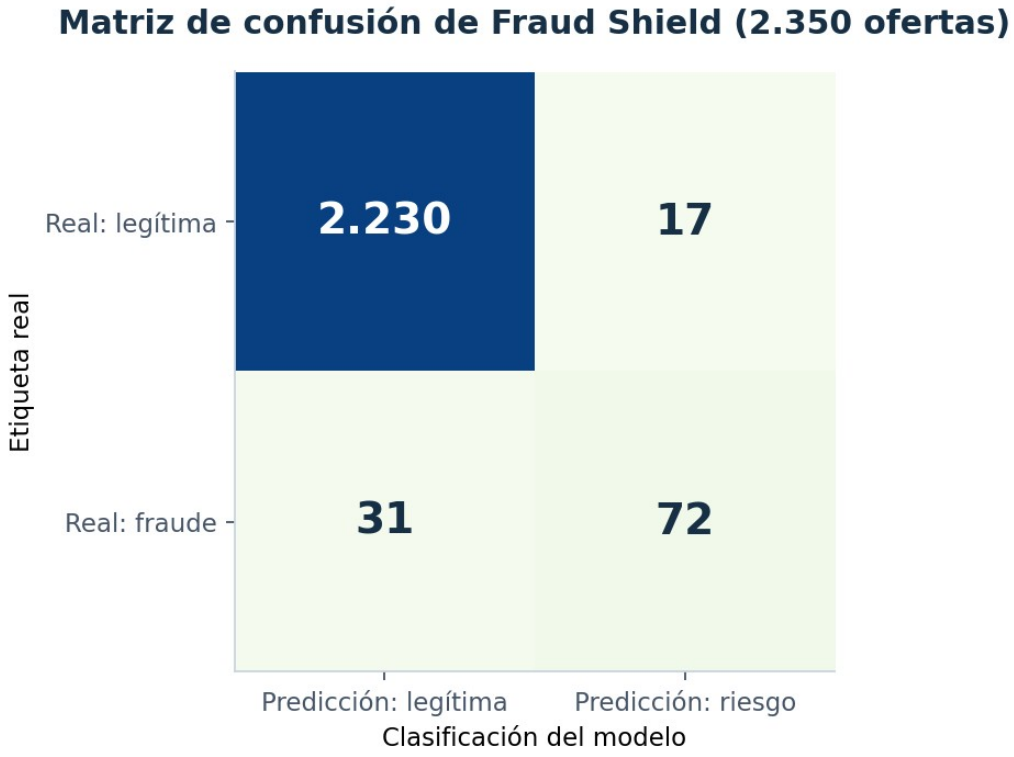
Tabla 5

Métricas del modelo de fraude

Métrica	Validación	Prueba
Precisión de fraude	0,8537	0,8090
Recall de fraude	0,6863	0,6990
F1 de fraude	0,7609	0,7500
Average Precision	0,7862	0,7958
Umbral	0,60	0,60

Figura 5

Matriz de confusión de Fraud Shield sobre 2.350 ofertas.



Nota. Elaboración propia a partir de reports/entrenamiento_fraude.json. Verdaderos positivos: 72; falsos negativos: 31; falsos positivos: 17; verdaderos negativos: 2.230.

En los 103 ejemplos fraudulentos del conjunto de prueba, el sistema identificó 72 y dejó sin detectar 31. Entre las 2.247 ofertas legítimas, marcó 17 como sospechosas. Los valores

demuestran utilidad como señal de alerta, pero no justifican decisiones automáticas ni permiten afirmar que una oferta es segura. El desbalance explica por qué no se usó accuracy como indicador central: clasificar casi todas las ofertas como legítimas podría producir una exactitud aparente elevada a costa de ignorar fraudes.

8. Integración de la aplicación web

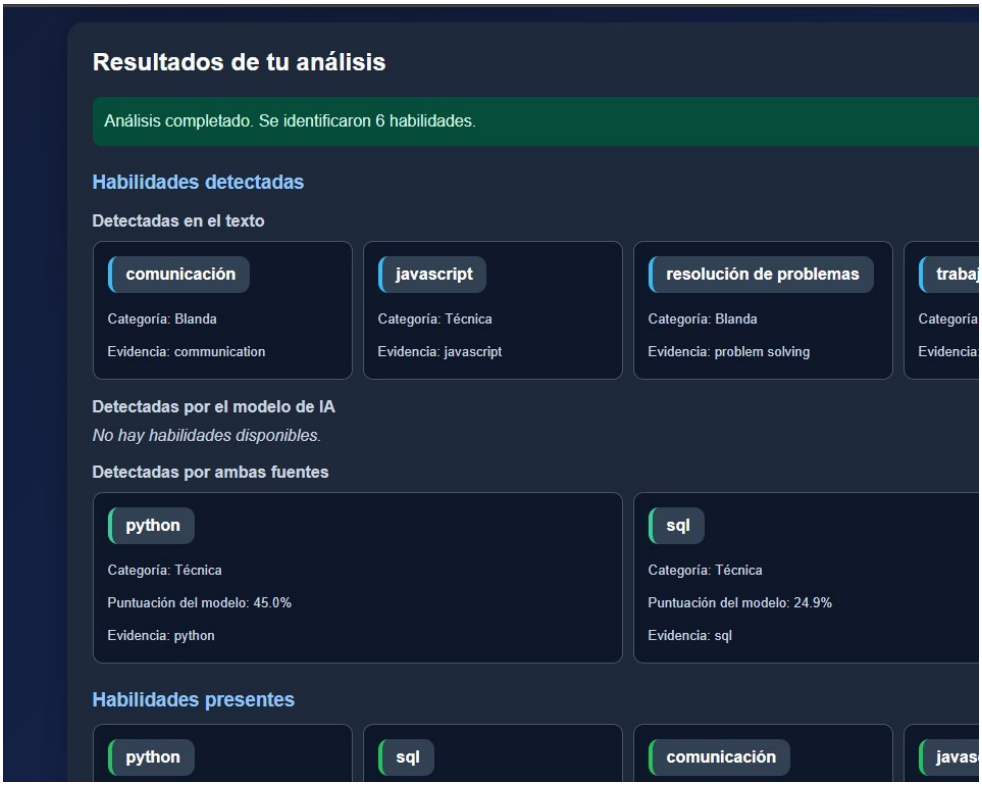
8.1. Arquitectura cliente-servidor

Sergio desarrolló la interfaz mediante HTML, CSS y JavaScript, mientras que Flask expone rutas que conectan el formulario con los servicios Python. El endpoint POST /api/analyze-profile recibe una descripción profesional y devuelve habilidades detectadas, categorías, brechas, compatibilidad y recomendaciones. El endpoint POST /api/fraud-risk recibe el texto de una oferta laboral y responde con un resultado orientativo de riesgo, su puntuación y una advertencia. El dashboard recupera datos agregados por medio de /api/dashboard-data.

La interfaz muestra las habilidades detectadas únicamente en texto, únicamente por el modelo y por ambas fuentes. Para compatibilidad laboral emplea coincidencias entre habilidades presentes y listas de competencias por familia; un valor de 24 % representa seis coincidencias entre veinticinco habilidades, no la probabilidad de conseguir empleo. La aplicación también presenta brechas de habilidades y recomendaciones basadas en las familias laborales priorizadas.

Figura 6

Análisis personalizado de habilidades en la interfaz de CareerTwin AI.

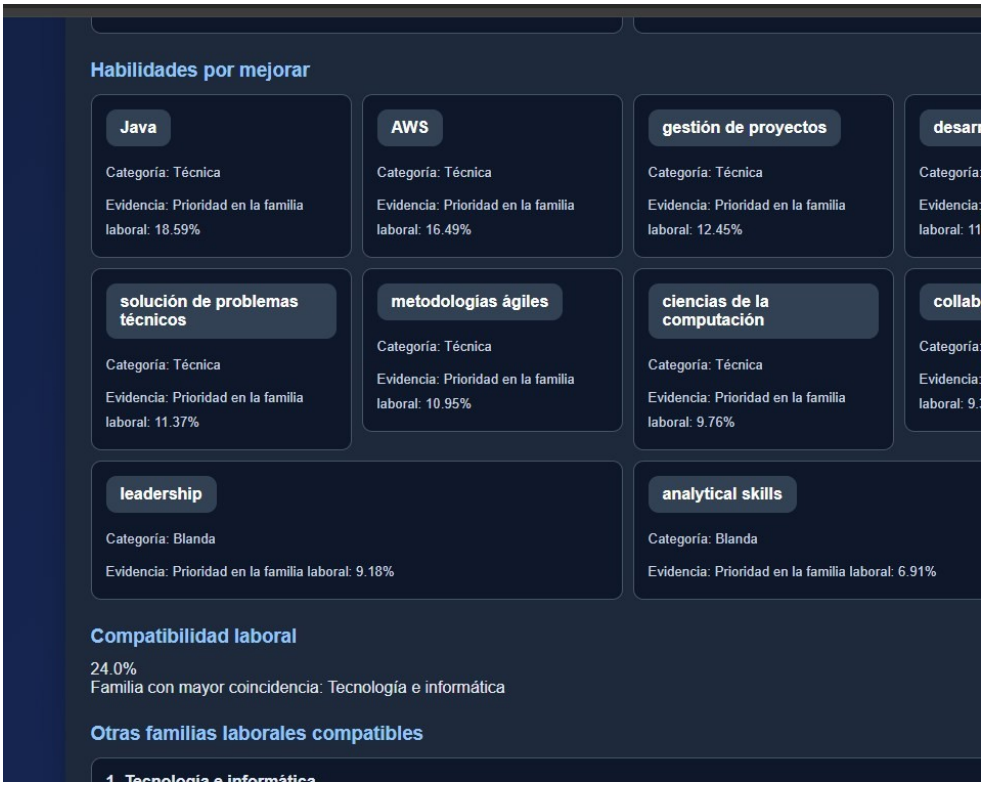


Nota. Captura del equipo, 9 de noviembre de 2026. Se identificaron seis competencias; Python y SQL figuran tanto por detección textual como por el modelo.

En el ejemplo mostrado, CareerTwin AI reconoció seis habilidades del perfil. Python y SQL aparecen respaldadas por la detección textual y la CNN, mientras que otras competencias se identificaron por coincidencia explícita en el texto. Esta distinción es relevante: el sistema combina una predicción estadística multietiqueta con reglas de extracción de habilidades, y la puntuación de la red no debe confundirse con una certificación ni con la probabilidad de contratación.

Figura 7

Brechas profesionales y coincidencias con familias laborales.



Nota. Captura del prototipo funcional. La familia Tecnología e informática obtuvo 24 % de coincidencia (6 de 25 habilidades).

8.2. Tres visualizaciones y sus fuentes

El panel general incluye las tres visualizaciones solicitadas en la rúbrica: (a) compatibilidad profesional para perfiles de ejemplo; (b) transiciones profesionales o similitud entre familias; y (c) volumen de ofertas por familia y habilidades más solicitadas. Los valores proceden de tablas agregadas y reglas del pipeline; la primera gráfica corresponde a perfiles ficticios de demostración, por lo que está señalada explícitamente como tal. No equivale a la predicción individual generada en el formulario superior.

Figura 8

Visualización 1: compatibilidad entre un perfil de ejemplo y familias laborales.



Nota. Captura aportada del dashboard desarrollado por Sergio. Los perfiles de ejemplo son demostrativos y no corresponden a usuarios reales.

Figura 9

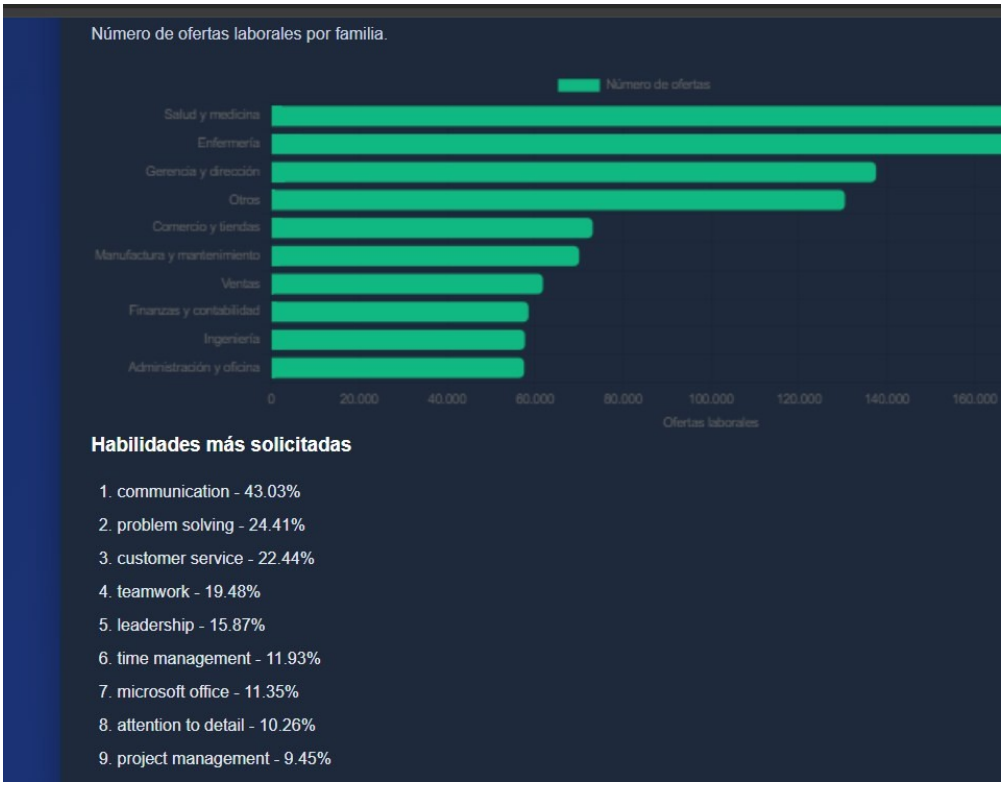
Visualización 2: similitud entre familias y transiciones profesionales.



Nota. Captura aportada del dashboard. Similaridad observada en relaciones de habilidades del corpus, sin garantía de cambio ocupacional.

Figura 10

Visualización 3: demanda de ofertas por familia y habilidades frecuentes.



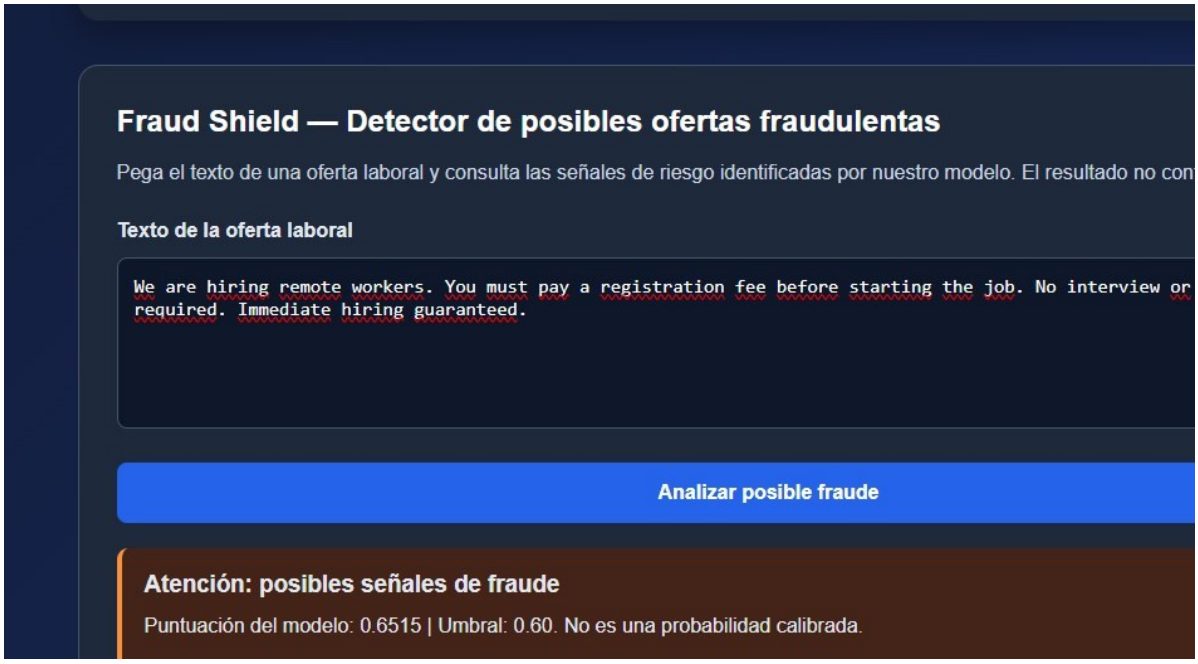
Nota. Captura aportada del dashboard; recuentos derivados de datos preparados del conjunto de LinkedIn.

8.3. Integración del formulario Fraud Shield

Se incorporó una sección para pegar el texto de una oferta y consultar el endpoint de fraude desde la misma página. En la prueba funcional capturada, el texto ficticio que solicitaba un pago previo al empleo recibió una puntuación de 0,6515 frente al umbral 0,60 y produjo la advertencia «Atención: posibles señales de fraude». Este caso demuestra la comunicación de extremo a extremo, no una validación estadística adicional; la clasificación puede cambiar con otro texto o modelo.

Figura 11

Interfaz integrada de Fraud Shield con una oferta ficticia de riesgo.



Nota. Captura del 9 de noviembre de 2026. Puntuación del modelo 0,6515 y umbral 0,60; el mensaje advierte que no se confirma fraude.

9. Resultados y validación

9.1. Verificación funcional

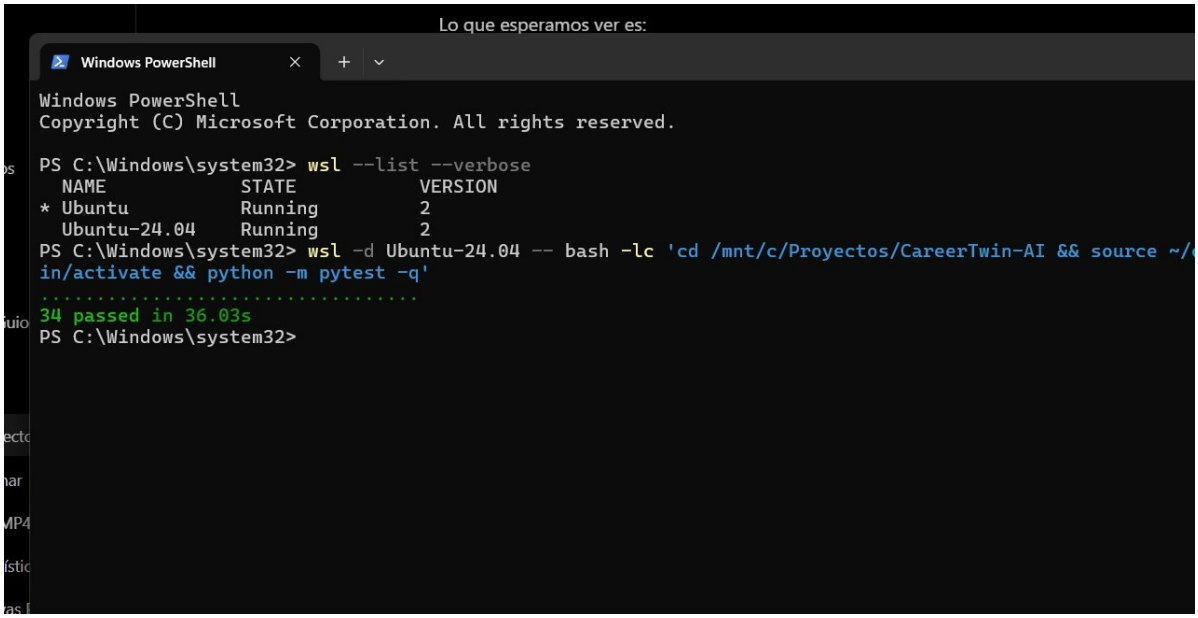
La integración se sometió a pruebas manuales sobre el formulario de perfiles, la generación de listas de habilidades y recomendaciones, las visualizaciones del dashboard y la respuesta del detector de fraude. La documentación se mantuvo en el repositorio y los cambios se incorporaron mediante ramas especializadas y Pull Requests, entre ellos las actualizaciones de datos y la integración final de la interfaz de fraude.

Como verificación final, en Ubuntu-24.04 se ejecutó python -m pytest -q sobre la versión sincronizada con main. El resultado registrado fue «34 passed in 36.03s». Este resultado

evidencia que la suite de pruebas disponible completó todas sus comprobaciones; no constituye una garantía de ausencia de defectos o de cumplimiento de requisitos productivos.

Figura 12

Resultado de la ejecución final de pruebas automatizadas.



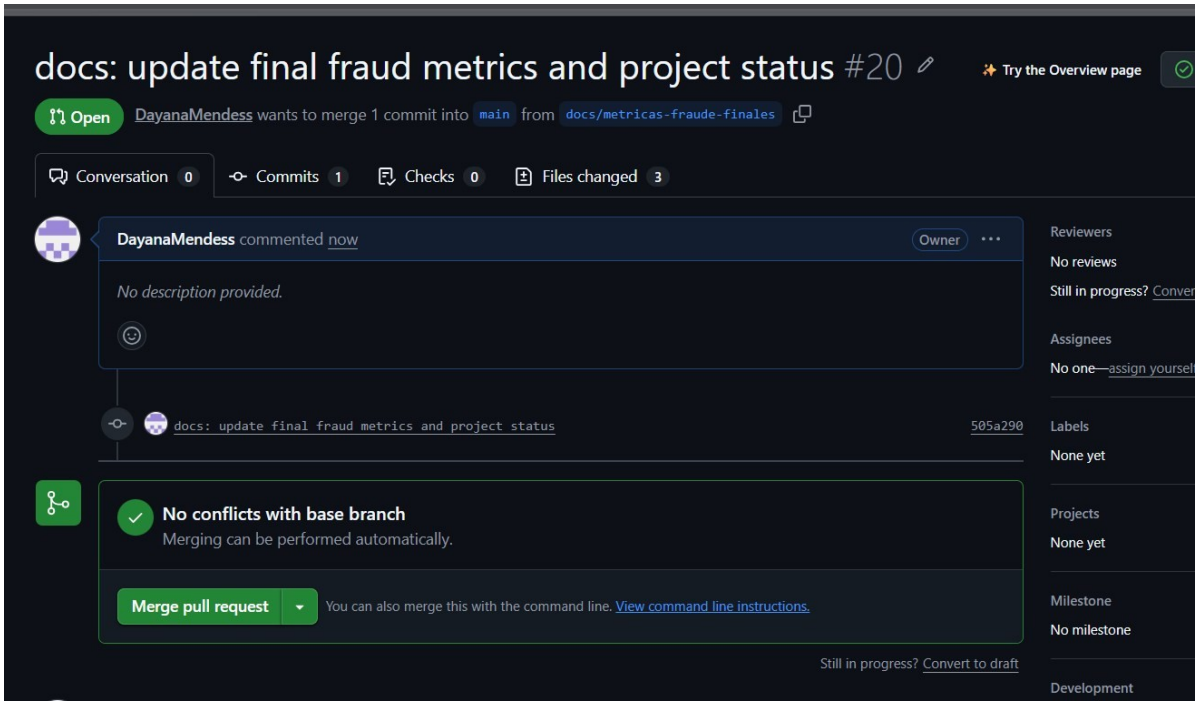
Nota. Captura del equipo: 34 pruebas completadas correctamente en 36,03 segundos desde Ubuntu-24.04.

9.2. Evidencia de colaboración y control de versiones

El repositorio público DayanaMendess/CareerTwin-AI conserva scripts, reportes JSON, documentos de ejecución, pruebas y la aplicación Flask. Los modelos entrenados (.keras y .joblib) se distribuyen por separado y no se incluyen en los commits. Esta separación evita incorporar binarios pesados en el historial del repositorio; para reproducir la inferencia desde una copia descargada es indispensable aportar dichos artefactos o entrenarlos nuevamente con los datos preparados.

Figura 13

Evidencia de revisión y merge de un Pull Request en GitHub.



Nota. Captura aportada del proceso colaborativo. El PR #20 actualizó documentación y métricas de fraude; la UI de fraude se integró posteriormente mediante PR #21.

10. Despliegue, reproducibilidad y limitaciones

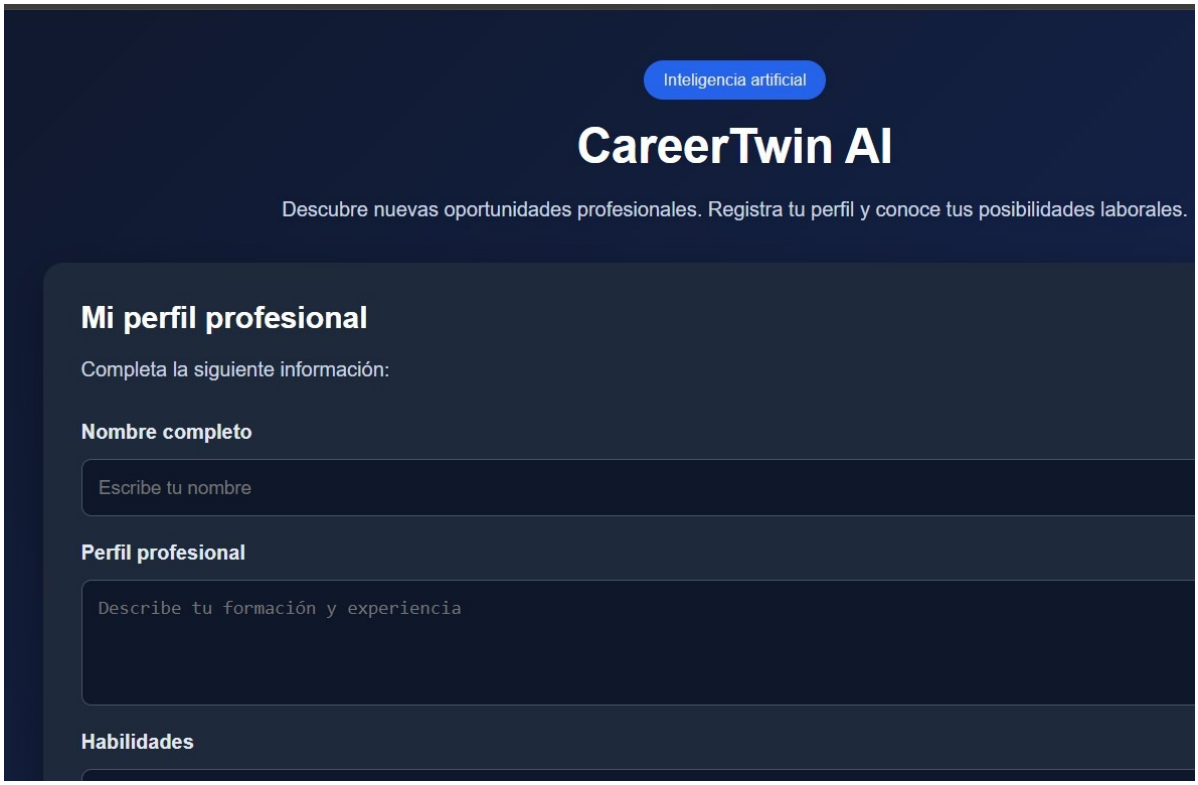
10.1. Ejecución y acceso remoto de demostración

El prototipo fue ejecutado en Windows con WSL Ubuntu 24.04, un entorno virtual de Python y Gunicorn mediante el comando «gunicorn --workers 1 --threads 2 --timeout 180 --bind 127.0.0.1:5000 app.main:app». Se utilizaron modelos locales bajo la carpeta models y se comprobó su respuesta desde el navegador y mediante consultas API. Para acceder desde otro dispositivo se habilitó Cloudflare Quick Tunnel, que expone temporalmente el servidor local a través de una URL HTTPS del dominio trycloudflare.com.

Este mecanismo permitió abrir la aplicación desde otro dispositivo y realizar análisis; sin embargo, no es un despliegue permanente en un servidor de producción. Depende de que permanezcan activos el computador, Gunicorn, cloudflared y la conexión a Internet. La rúbrica del profesor solicita expresamente un servidor de producción (Render, AWS, Google Cloud u otro), por lo cual esta implementación debe presentarse con transparencia como solución temporal de demostración, no como cumplimiento integral del punto 5.

Figura 14

CareerTwin AI accesible mediante enlace HTTPS temporal de Cloudflare.



Nota. Captura aportada durante las pruebas del túnel público. La URL es temporal y puede cambiar al reiniciar cloudflared.

10.2. Reproducibilidad y seguridad

El repositorio incluye el código fuente y los archivos de configuración y reportes, mientras que los CSV originales, matrices procesadas y modelos exportados se mantienen fuera de Git. La documentación de ejecución contempla la instalación de dependencias, rutas y procedimientos de validación. Para reproducir la aplicación son necesarios los archivos career_skills.keras y fraud_detector.joblib, compatibles con las versiones de las bibliotecas instaladas. No se recomienda exponer el servidor de desarrollo de Flask con depuración activada.

10.3. Limitaciones identificadas

Las ofertas de LinkedIn se concentran en países angloparlantes en 2024 y no representan de forma directa el mercado de trabajo actual del Huila o Colombia.

La CNN tiene un Macro-F1 bajo y muchas etiquetas poco frecuentes; sus salidas son exploratorias y requieren más entrenamiento, revisión de etiquetas y evaluación externa.

La compatibilidad del perfil es una medida de cobertura de habilidades por familia, no una probabilidad de contratación ni una medición de aptitudes personales.

Fraud Shield fue evaluado sobre EMSCAD, un corpus histórico de otra plataforma y periodo; la sensibilidad puede disminuir ante fraude nuevo o cambios de dominio.

Las puntuaciones de los modelos no son probabilidades calibradas; tampoco constituyen certificación de habilidades o evidencia jurídica de fraude.

Los perfiles de demostración de las gráficas generales son ficticios; el simulador de escenarios hipotéticos no cuenta todavía con una experiencia completa para usuario final.

El acceso remoto verificado utiliza un túnel temporal y no equivale a despliegue permanente con operación, almacenamiento y monitoreo de producción.

11. Conclusiones

CareerTwin AI alcanzó el objetivo central de construir una solución NLP que ejecuta una red neuronal real y comunica sus resultados a través de Flask. Se documentaron las etapas de auditoría, normalización, generación de etiquetas, división sin fuga, arquitectura neuronal, entrenamiento de 30 épocas, selección de umbral y evaluación final. Las métricas obtenidas muestran que el módulo de habilidades funciona técnicamente, pero su calidad predictiva necesita mejoras antes de un uso real de orientación profesional.

El módulo Fraud Shield amplió el alcance del prototipo y demostró el valor de preparar datos con atención a duplicados, desbalance y compatibilidad entre entrenamiento e inferencia. Su F1 de 0,7500 para fraude y la matriz de confusión ofrecen una evaluación más apropiada que la exactitud global en un dataset con pocos positivos; aun así, la herramienta debe mantenerse como advertencia de riesgo con validación humana.

La integración colaborativa mediante GitHub y los 34 casos automatizados aprobados aportan trazabilidad y respaldo funcional. La interfaz presenta resultados de usuario, brechas y tres visualizaciones del mercado laboral; se identificó además la necesidad de distinguir con claridad los perfiles de demostración. La exposición del prototipo por Cloudflare permitió comprobaciones externas, pero no resuelve el alojamiento productivo solicitado por la rúbrica.

12. Recomendaciones

Se recomienda priorizar en una siguiente iteración la validación con datos nuevos y perfiles en español, el aumento de cobertura del vocabulario, experimentos de modelos preentrenados o ponderación por clases, la calibración de puntuaciones, un simulador interactivo completo y el despliegue en infraestructura de producción con acceso controlado, monitoreo y prácticas de privacidad.

Referencias

Asaniczka. (2024). 1.3M LinkedIn jobs & skills (2024) [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/asaniczka/1-3m-linkedin-jobs-and-skills-2024

Bansal, S. (s. f.). Real or fake: Fake job posting prediction [Conjunto de datos, EMSCAD]. Kaggle. https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction

Chollet, F. (2021). Deep learning with Python (2.ª ed.). Manning.

Corporación Universitaria del Huila. (2026). Proyecto final de Inteligencia Artificial [Guía de la asignatura]. CORHUILA.

DayanaMendess, equipo CareerTwin AI. (2026). CareerTwin-AI [Código fuente y reportes técnicos]. GitHub. https://github.com/DayanaMendess/CareerTwin-AI

Keras. (s. f.). Keras: The deep learning API. https://keras.io/

Kim, Y. (2014). Convolutional neural networks for sentence classification. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, 1746–1751. https://doi.org/10.3115/v1/D14-1181

Pallets. (s. f.). Flask documentation. https://flask.palletsprojects.com/

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

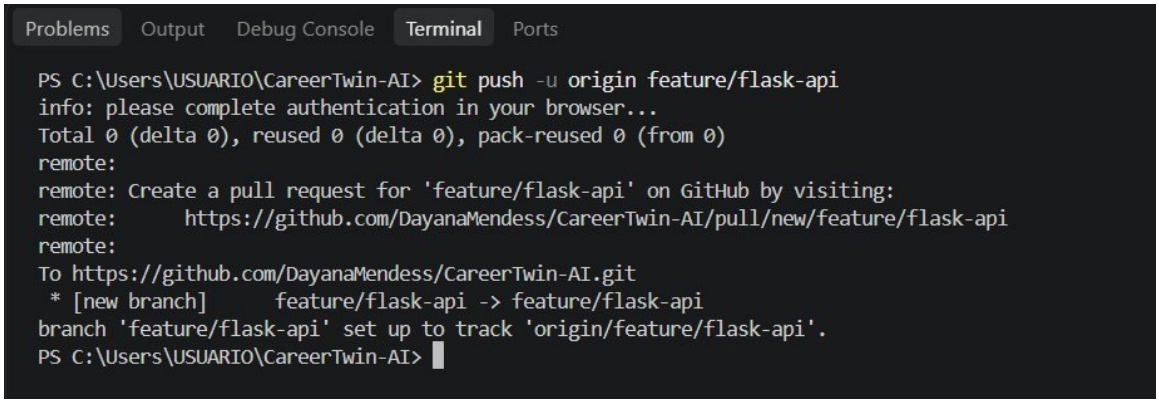
TensorFlow. (s. f.). TensorFlow: End-to-end machine learning platform. https://www.tensorflow.org/

Anexo A. Evidencias visuales del desarrollo

Se presenta una selección de capturas aportadas durante el desarrollo y la integración. Para evitar duplicidades, las imágenes repetitivas de terminal y editor no se reproducen íntegramente; se seleccionan evidencias representativas por etapa. Los textos de ChatGPT visibles incidentalmente en alguna captura son contextuales y no constituyen resultados medidos del sistema.

Figura 15

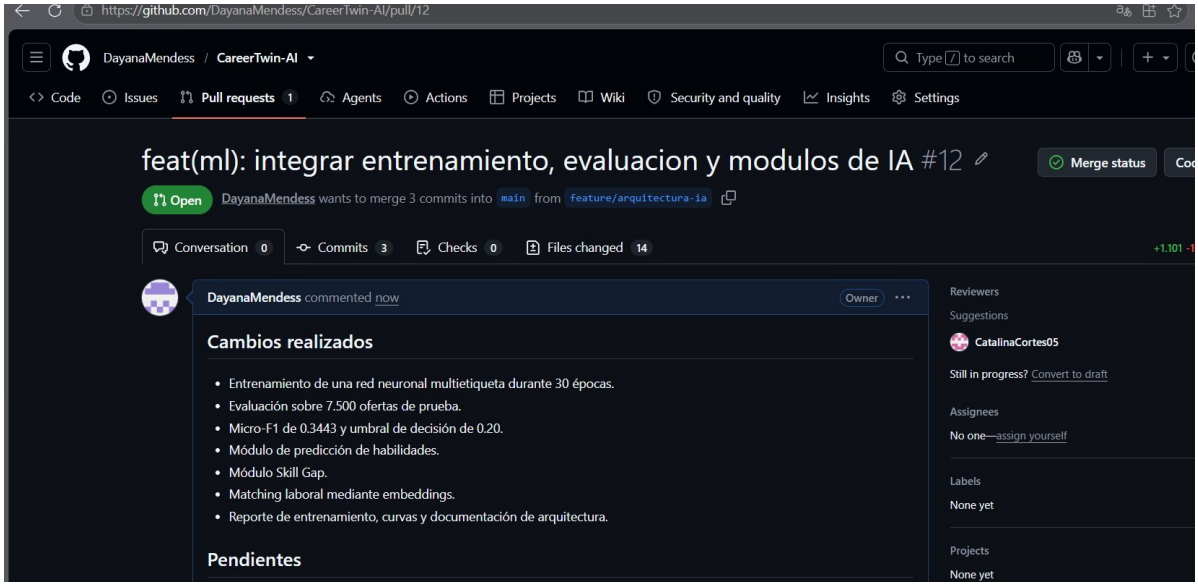
Evidencia de trabajo colaborativo: publicación de una rama de Flask en GitHub.



Nota. Captura aportada por el equipo durante la fase inicial de desarrollo de backend e integración de Sergio.

Figura 16

Evidencia del entorno de desarrollo y revisión de código.



Nota. Captura aportada de VS Code y del proceso de integración colaborativo; el repositorio conserva el historial completo.

Figura 17

Recomendaciones profesionales generadas para el perfil de prueba.



Nota. Captura del prototipo: Java, AWS y gestión de proyectos aparecen como habilidades a fortalecer, de forma orientativa.

Anexo B. Comprobaciones para la sustentación

Antes de exponer, el equipo debe confirmar: que main está sincronizada; que los dos modelos entrenados están presentes en models; que Gunicorn inicia en Ubuntu-24.04 y la interfaz responde en 127.0.0.1:5000; que ambas funcionalidades permiten enviar textos ficticios y mostrar resultados; que el dashboard carga tres gráficas; que python -m pytest -q termina sin fallos; y que el enlace Cloudflare funciona desde una red externa mientras los procesos permanecen activos.